

Siguiendo estrictamente la arquitectura establecida, avanzaremos al desarrollo del módulo del mundo (World), encargado de gestionar la lógica y el renderizado del mapa. Para ello, crearemos la clase GameMap y la integraremos en nuestro estado de simulación (GameState). A continuación, presento la entrega del escenario base.

1. Estructura de Carpetas Actualizada

Se crea el nuevo archivo dentro del módulo world, tal como se diseñó en la arquitectura original.

Archivos Creados:

- `src/world/__init__.py` (Archivo vacío para inicializar el paquete Python).
- `src/world/map.py` (Lógica visual y espacial del escenario).

Archivos Modificados:

- `src/core/state_manager.py` (Se integra el mapa dentro del GameState).

```
Plaintext
el_ultimo_bastion/
├── config/
│   └── settings.json
├── src/
│   ├── core/
│   │   ├── __init__.py
│   │   ├── game_manager.py
│   │   └── state_manager.py [MODIFICADO]
│   ├── world/ [NUEVO MÓDULO]
│   │   ├── __init__.py [NUEVO]
│   │   └── map.py [NUEVO]
│   └── __init__.py
├── main.py
└── requirements.txt
```

2. Código Completo

`src/world/map.py` (Nuevo)

```
import pygame
```

```
class GameMap:
```

```
    """
```

```
Gestiona el escenario base del videojuego, incluyendo el fondo,  
las rutas de los enemigos, el bastión y las zonas de construcción.  
    """
```

```
    def __init__(self, screen_width: int, screen_height: int):  
        self.width = screen_width  
        self.height = screen_height  
  
        # Paleta de colores (Placeholder hasta integrar sprites)  
        self.color_bg = (34, 47, 34) # Terreno (Verde militar oscuro)  
        self.color_path = (139, 115, 85) # Camino de tierra (Marrón)  
        self.color_bastion = (70, 70, 90) # Bastión (Piedra oscura)  
        self.color_build_zone = (50, 70, 50) # Zona construible (Verde claro opaco)  
        self.color_border = (100, 150, 100) # Borde de zona construible  
  
        # 1. Definición del Bastión (Ubicado a la derecha de la pantalla)  
        bastion_size = 120  
        self.bastion_rect = pygame.Rect(  
            self.width - 150,  
            (self.height // 2) - (bastion_size // 2),  
            bastion_size,  
            bastion_size  
        )  
  
        # 2. Definición del Camino (Waypoints / Nodos)  
        # Los enemigos seguirán esta secuencia de coordenadas exactas.  
        self.waypoints = [  
            (0, 120), # Punto de spawn (Izquierda)  
            (350, 120), # Avanza a la derecha  
            (350, 550), # Baja  
            (850, 550), # Avanza a la derecha  
            (850, self.height // 2), # Sube a la altura del bastión  
            (self.bastion_rect.left, self.height // 2) # Impacta el bastión  
        ]  
  
        # Generación de la geometría visual del camino  
        self.path_width = 60  
        self.path_rects = self._generate_path_rects()  
  
        # 3. Definición de Zonas Construibles  
        # Áreas donde el jugador podrá colocar torres sin obstruir el camino.  
        self.buildable_zones = [
```

```

pygame.Rect(100, 200, 200, 250),
pygame.Rect(450, 100, 300, 150),
pygame.Rect(450, 350, 300, 150),
pygame.Rect(950, 150, 250, 150),
pygame.Rect(950, 450, 250, 150)
]

```

```

def _generate_path_rects
    """Calcula los rectángulos visuales que conectan los waypoints del camino."""
    rects = []
    pw = self.path_width
    for i in range(len(self.waypoints) - 1):
        x1, y1 = self.waypoints[i]
        x2, y2 = self.waypoints[i+1]

        if x1 == x2: # Tramo Vertical
            top = min(y1, y2) - pw // 2
            left = x1 - pw // 2
            width = pw
            height = abs(y2 - y1) + pw
            rects.append(pygame.Rect(left, top, width, height))
        elif y1 == y2: # Tramo Horizontal
            left = min(x1, x2) - pw // 2
            top = y1 - pw // 2
            width = abs(x2 - x1) + pw
            height = pw
            rects.append(pygame.Rect(left, top, width, height))
    return rects

```

```

def update          float
    """Actualiza lógicas de entorno (animaciones ambientales futuras)."""
    pass

```

```

def draw
    # Dibujar Fondo
    screen.fill(self.color_bg)

    # Dibujar Zonas Construibles
    for zone in self.buildable_zones:
        pygame.draw.rect(screen, self.color_build_zone, zone)
        pygame.draw.rect(screen, self.color_border, zone, 2) # Borde

    # Dibujar Camino

```

```
for rect in self.path_rects:
    pygame.draw.rect(screen, self.color_path, rect)

# Dibujar Bastión
pygame.draw.rect(screen, self.color_bastion, self.bastion_rect)
pygame.draw.rect(screen, (30, 30, 40), self.bastion_rect, 4) # Borde fuerte

# Etiqueta del Bastión
font = pygame.font.SysFont("Arial", 22, bold=True)
text = font.render("BASTIÓN", True, (220, 220, 220))
text_rect = text.get_rect(center=self.bastion_rect.center)
screen.blit(text, text_rect)
```

src/core/state_manager.py (Modificación en GameState)

Nota: Solo se muestra la clase GameState actualizada; el resto del archivo se mantiene exactamente igual que en tu versión anterior.

```
from src.world.map import GameMap # IMPORTANTE: Nuevo import al inicio del archivo

class GameState(State):
    """Estado: Simulación principal del juego"""
    def __init__(
        self,
        super().__init__(state_manager)
        self.game_map = None

    def on_enter
        # Se inicializa el mapa dinámicamente tomando la resolución actual
        if not self.game_map:
            screen = pygame.display.get_surface()
            self.game_map = GameMap(screen.get_width(), screen.get_height())

    def handle_events
        for event in events:
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_ESCAPE:
                    self.manager.change_state("PAUSE")
```

```

        elif event.key == pygame.K_g:
            self.manager.change_state("GAME_OVER")

def update
    # Delegar actualización temporal al mapa
    if self.game_map:
        self.game_map.update(dt)

def draw
    # 1. Delegar el renderizado completo del fondo y entidades estáticas al mapa
    if self.game_map:
        self.game_map.draw(screen)

    # 2. Renderizar UI superpuesta (HUD temporal)
    font = pygame.font.SysFont("Arial", 20, bold=True)
    ui_text = font.render("[ESC] Pausar | [G] Simular Game Over", True, (255, 255, 255))

    # Fondo oscuro semitransparente para el texto
    bg_surface = pygame.Surface((ui_text.get_width() + 20, ui_text.get_height() + 10))
    bg_surface.set_alpha(150)
    bg_surface.fill((0, 0, 0))

    screen.blit(bg_surface, (15, 15))
    screen.blit(ui_text, (25, 20))

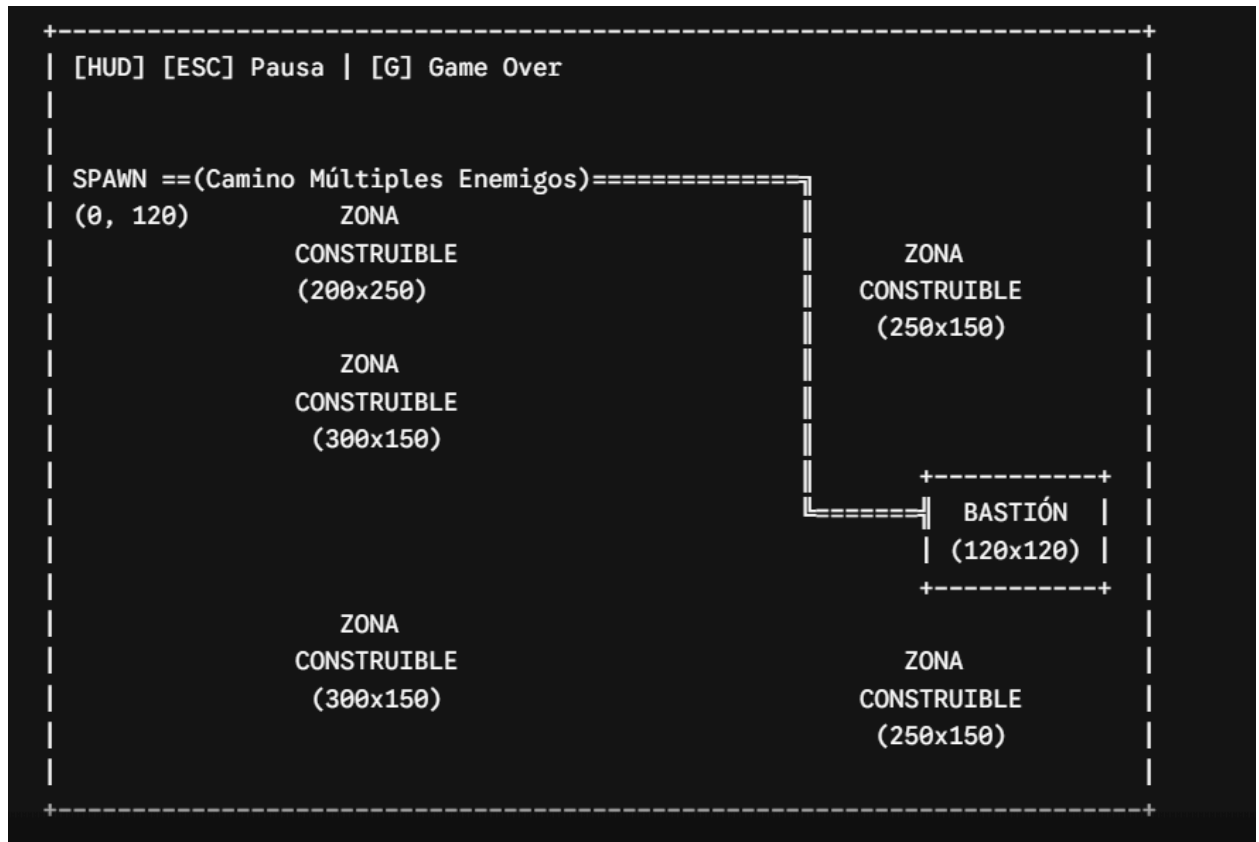
```

3. Explicación Técnica

- **Geometría Matemática de Rutas (_generate_path_rects):** En lugar de dibujar rectángulos estáticos a mano (lo cual rompe el principio de mantenimiento), he programado un sistema que lee los waypoints (nodos por donde caminarán los enemigos) y genera dinámicamente los trazos de los caminos. Esto garantiza que cuando programemos los Goblins, estos caminarán exactamente sobre el mismo suelo que se ha dibujado, eliminando bugs de colisiones visuales.
- **Aislamiento de Responsabilidad (Single Responsibility Principle):** La clase GameState no sabe cómo está dibujado el mapa ni qué color tiene el bastión. Simplemente llama a `self.game_map.draw(screen)`. Esto es vital para poder cargar múltiples mapas distintos en el futuro (Nivel 1, Nivel 2, etc.) pasando diferentes archivos de configuración a GameMap sin reescribir la lógica de estados.
- **Zonas Construibles (pygame.Rect):** Se definieron rectángulos perimetrales (`buildable_zones`). En futuras etapas, cuando implementemos el ratón (mouse), se utilizará el método `collidepoint()` de Pygame contra estos rectángulos para determinar si una torre puede ser plantada legalmente.

4. Captura Conceptual del Resultado Esperado

Al ejecutar el juego y presionar *ENTER* en el menú, el motor renderizará una escena de 1280x720 píxeles. Visualmente estará organizada mediante este layout:



Instrucciones de Integración:

1. Crea el archivo `map.py` en la carpeta `src/world/`.
2. Crea un archivo vacío `__init__.py` junto a `map.py`.
3. Actualiza **exclusivamente** la clase `GameState` en `src/core/state_manager.py` tal y como se muestra (asegurándote de añadir el `import` al principio del archivo).
4. Ejecuta `python main.py` desde la raíz.